

SGN – Assignment #1

Andrea D'Uva, 942644

1 Periodic orbit

Exercise 1

Consider the 3D Sun–Earth Circular Restricted Three-Body Problem with $\mu = 3.0359 \times 10^{-6}$.

- 1) Find the x -coordinate of the Lagrange point L_2 in the rotating, adimensional reference frame with at least 10-digit accuracy.

Solutions to the 3D CRTBP satisfy the symmetry

$$\mathcal{S} : (x, y, z, \dot{x}, \dot{y}, \dot{z}, t) \rightarrow (x, -y, z, -\dot{x}, \dot{y}, -\dot{z}, -t).$$

Thus, a trajectory that crosses perpendicularly the $y = 0$ plane twice is a periodic orbit.

- 2) Given the initial guess $\mathbf{x}_0 = (x_0, y_0, z_0, v_{x0}, v_{y0}, v_{z0})$, with

$$\begin{aligned} x_0 &= 1.008296144180133 \\ y_0 &= 0 \\ z_0 &= 0.001214294450297 \\ v_{x0} &= 0 \\ v_{y0} &= 0.010020975499502 \\ v_{z0} &= 0 \end{aligned}$$

Find the periodic halo orbit that passes through z_0 ; that is, develop the theoretical framework and implement a differential correction scheme that uses the STM either approximated through finite differences or achieved by integrating the variational equation.

The periodic orbits in the CRTBP exist in families. These can be computed by continuing the orbits along one coordinate, e.g., z_0 . This is an iterative process in which one component of the state is varied, while the other components are taken from the solution of the previous iteration.

- 3) By gradually increasing z_0 and using numerical continuation, compute the families of halo orbits until $z_0 = 0.0046$.

(8 points)

1) In the 3D CRTBP it's usually convenient to work with a rotating frame, centered at the baricenter of the system and having the x axis always aligned with the two masses m_1 and m_2 , the z axis perpendicular to the planets orbital plane and the y axis completing the basis. The equations of motion can then be compactly written as follows:

$$\begin{cases} \ddot{x} - 2\dot{y} = \frac{\partial U}{\partial x} \\ \ddot{y} + 2\dot{x} = \frac{\partial U}{\partial y} \\ \ddot{z} = \frac{\partial U}{\partial z} \end{cases} \quad \text{being the potential function } U = \frac{1}{2}(x^2 + y^2 + z^2) + \frac{1-\mu}{r_1} + \frac{\mu}{r_2} \quad (1)$$

The system of equations 1 is non-dimensional and depends only on the mass ratio parameter $\mu = \frac{m_2}{m_1 + m_2}$. It can be rewritten in the usual canonical form by defining the state variable $\mathbf{x} = [x, y, z, v_x, v_y, v_z]^T$, and the distances $r_1 = \sqrt{(x + \mu)^2 + y^2 + z^2}$, $r_2 = \sqrt{(x + \mu - 1)^2 + y^2 + z^2}$; its expression is reported below.

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) \quad \text{which explicitly is} \quad \begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ \dot{z} = v_z \\ \dot{v}_x = 2v_y + x - \frac{(1-\mu)(x+\mu)}{r_1^3} - \frac{\mu}{r_2^3}(x+\mu-1) \\ \dot{v}_y = -2v_x + y - \frac{(1-\mu)}{r_1^3}y - \frac{\mu}{r_2^3}y \\ \dot{v}_z = -\frac{(1-\mu)}{r_1^3}z - \frac{\mu}{r_2^3}z \end{cases} \quad (2)$$

Equation 2 admits equilibrium points whose state is obtained by the solution of the condition $\mathbf{f}(\mathbf{x}_e) = 0$, meaning that their relative position, with respect to both the Earth and the Sun, is kept constant. Of specific importance are the so-called Collinear points L_1 , L_2 , and L_3 , which are all characterized by having null coordinates, except only for x . These three values are obtained by solving the implicit nonlinear equation:

$$\dot{v}_x = U_x(x, 0, 0) = \frac{\partial U(x)}{\partial x} \quad (3)$$

The three roots of Equation 3 depend on the value of μ and their location becomes more and more unbalanced as its value increases. For instance, in the Earth-Sun system, the L_1 and L_2 points are located extremely close to the Earth, labelled here m_2 , as depicted in Figure 1. The L_2 point in particular is the farthestmost point among the three ones, for which then the Earth always stands between itself and the Sun.

In order to ensure a result accurate enough for the computation of its x coordinates, a tolerance of 10^{-12} has been set on the MATLAB function `fzero` by means of the optional parameter `TolX`. There is also the `fsolve` tool, but it is usually exploited to solve systems of nonlinear equations only and is not recommended being applied on scalar problems. Having defined by visual inspection the search interval as $[1.0016 \ 1.02]$, the value of the resulting L_2 x coordinate is reported below:

$$x_{L_2} = 1.010070187506$$

The State Transition Matrix is defined as the Jacobian matrix of the flow of the ODE $\varphi(\mathbf{x}, t_0; t)$ and allows to map a displacement from the initial state into a displacement at final time: $\delta\mathbf{x} = \Phi(t_0, t) \delta\mathbf{x}_0$.

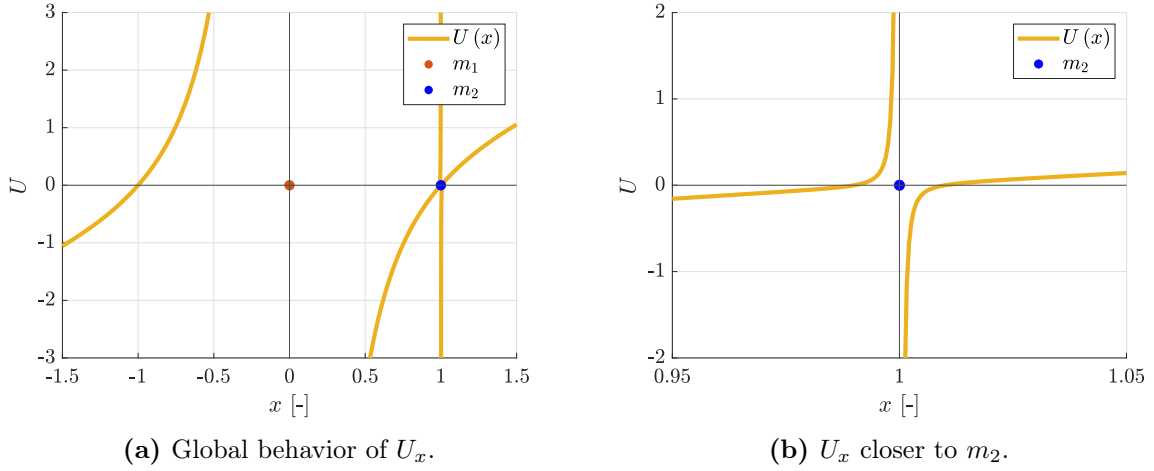


Figure 1: Graph of the function $\frac{\partial U(x)}{\partial x}$.

$$\begin{cases} \dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}) \\ \dot{\Phi}(t_0, t) &= A\Phi(t_0, t) \\ \mathbf{x}(t_0) &= \mathbf{x}_0 \\ \Phi(t_0, t_0) &= I \end{cases} \quad \text{being} \quad A = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} 0 & I \\ V & 2\Omega \end{bmatrix} \quad (4)$$

where the four blocks of the 6×6 matrix A are:

$$I = 3 \times 3 \text{ identity matrix, } \Omega = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad \text{and} \quad V = V^T = \begin{bmatrix} \frac{\partial \dot{v}_x}{\partial x} & \frac{\partial \dot{v}_x}{\partial y} & \frac{\partial \dot{v}_x}{\partial z} \\ \frac{\partial \dot{v}_y}{\partial x} & \frac{\partial \dot{v}_y}{\partial y} & \frac{\partial \dot{v}_y}{\partial z} \\ \frac{\partial \dot{v}_z}{\partial x} & \frac{\partial \dot{v}_z}{\partial y} & \frac{\partial \dot{v}_z}{\partial z} \end{bmatrix}$$

$$\begin{aligned} V_{1,1} &= 1 - (1 - \mu)/r_1^3 + 3(1 - \mu)(x + \mu)^2/r_1^5 - \mu/r_2^3 + 3\mu(x + \mu - 1)^2/r_2^5 \\ V_{2,2} &= 1 - (1 - \mu)/r_1^3 + 3(1 - \mu)y^2/r_1^5 - \mu/r_2^3 + 3\mu y^2/r_2^5 \\ V_{3,3} &= -(1 - \mu)/r_1^3 + 3(1 - \mu)z^2/r_1^5 - \mu/r_2^3 + 3\mu z^2/r_2^5 \\ V_{1,2} &= V_{2,1} = 3y(1 - \mu)(x + \mu)/r_1^5 + 3\mu(x + \mu - 1)y/r_2^5 \\ V_{1,3} &= V_{3,1} = 3z(1 - \mu)(x + \mu)/r_1^5 + 3\mu(x + \mu - 1)z/r_2^5 \\ V_{2,3} &= V_{3,2} = 3yz(1 - \mu)/r_1^5 + 3\mu yz/r_2^5 \end{aligned}$$

with:

The variational equation 4 consists of 42 scalar ODEs to be numerically integrated at each time to compute the state and the STM. To prove the correctness of the numerical integrator the Jacobi constant has been used: Figure 2 reports the difference between the value obtained at each integration step and the exact one, with respect to the initial state $\mathbf{x}_0$ and over a time interval $[0 \ 1.5\pi]$. Being the two values always almost identical, the integrator has been deemed accurate enough.

2) These two tools are then exploited in the implementation of the differential corrector scheme, that allows to find the periodic halo orbit associated to the initial state $\mathbf{x}_0$. The method can be described as follows:

1. Integrate the variational equation from the initial state until the condition $y = 0$ is met. This can be done by means of MATLAB `event` locator routine, available when using numerical integrators.
2. To have a periodic orbit it is required to have both $v_x = 0$ and $v_z = 0$ when the event occurs. If it is not the case a correction has to be made on the initial coordinates x and v_y .

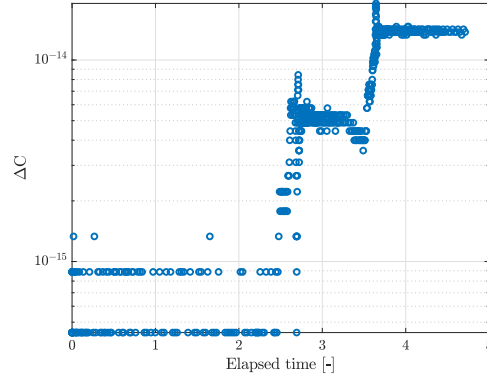


Figure 2: Jacobi constant behavior.

3. From the relation $\Delta \mathbf{x} = \Phi(t_0, t) \Delta \mathbf{x}_0$ retrieve the fourth and sixth equation:

$$\begin{cases} \Delta v_x^f = \Phi_{4,1} \Delta x^0 + \Phi_{4,5} \Delta v_y^0 \\ \Delta v_z^f = \Phi_{6,1} \Delta x^0 + \Phi_{6,5} \Delta v_y^0 \end{cases} \quad (5)$$

which by setting $\Delta v_x^f = -v_x^e$, $\Delta v_z^f = -v_z^e$ is a linear system in the unknown Δx^0 and Δv_y^0 .

4. Solve the linear system and update the initial state with the correction values.
5. Iterate until a tolerance value is met. The tolerance can be defined either on the y difference between two iterations or on v_x^e , v_z^e .

Table 1 lists the coordinates of the state found with the scheme mentioned above, that defines a periodic halo orbit around the L_2 point. Figure 3 graphically shows the result; to be noted that the numerical integration needs to be done only on the first half of the orbit: once the correct event is located, the second half of the trajectory comes from symmetry.

Table 1: Initial state obtained starting from $\mathbf{x}_0$.

x	y	z	v_x	v_y	v_z	$\frac{T}{2}$
1.00824790272	0	0.0012142944502	0	0.01033603458073	0	1.55014757267

3) To find the other halo orbits adjacent to the one obtained before, a scheme based on numerical continuation has been implemented:

1. Define a set of z coordinates inside the interval starting from the value reported in Table 1 until $z_0 = 0.0046$ and an initial state associated to the first value.
2. Using the differential corrector, retrieve the actual state that leads to a halo orbit with the fixed value of z .
3. Take the initial guess for the state corresponding to the next iterate as the result of the current step.

In this way for each z the associated x and v_y coordinates that generate a periodic orbit are obtained, and Figure 4 shows the results for 15 different z values: as it increases, the orbital size grows as well. To make this technique work however, the samples in the z interval should be dense enough, for instance in our case $N > 10$: if else, when switching to the next iteration, the initial guess may lead to a divergent trajectory.

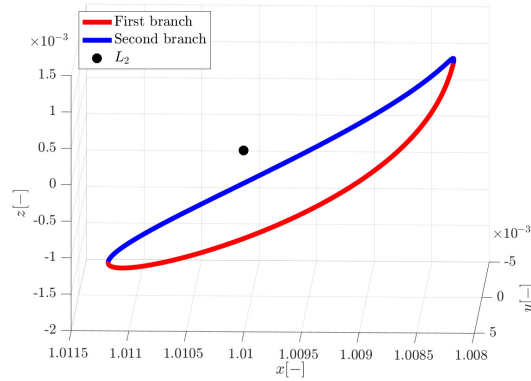
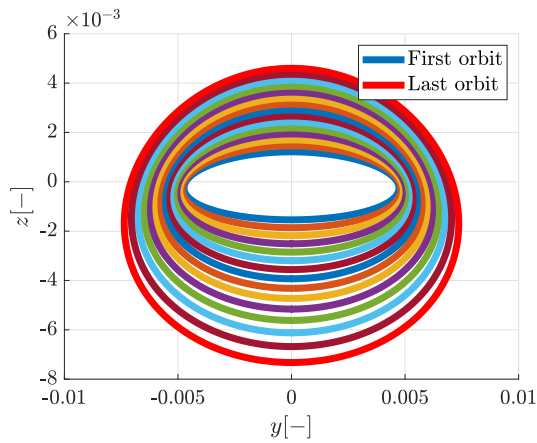
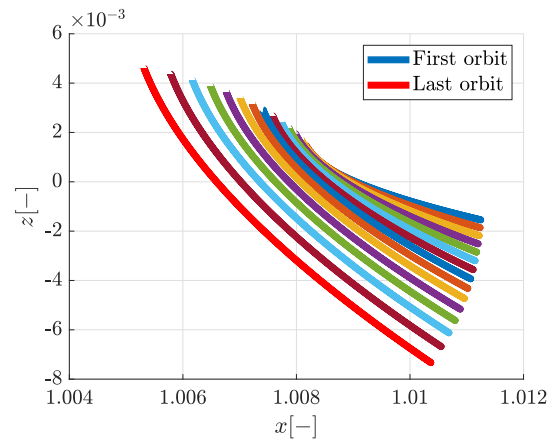


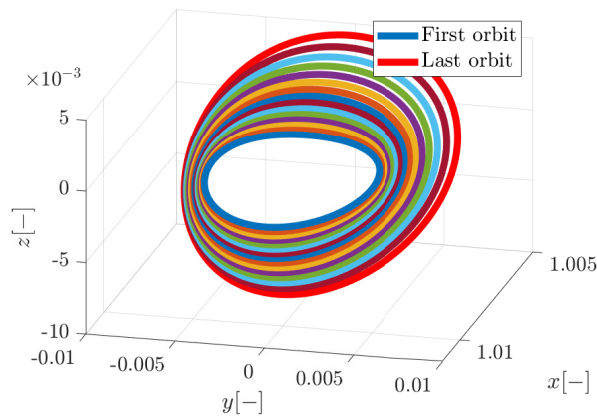
Figure 3: Period halo orbit found starting from $\mathbf{x}_0$, represented in the rotating frame.



(a) (y, z) plane trajectory.



(b) (x, z) plane trajectory.



(c) 3D trajectory.

Figure 4: $N = 15$ halo orbits represented in the rotating frame.

2 Impulsive guidance

Exercise 2

Consider the two-impulse transfer problem stated in Section 3.1 (Topputo, 2013)*.

- 1) Using the procedure in Section 3.2, produce a first guess solution using $\alpha = 1.5\pi$, $\beta = 1.41$, $\delta = 7$, and $t_i = 0$. Plot the solution in both the rotating frame and Earth-centered inertial frame (see Appendix 1 in (Topputo, 2013)).
- 2) Considering the first guess in 1) and using $\{\mathbf{x}_i, t_i, t_f\}$ as variables, solve the problem in Section 3.1 with simple shooting in the following cases
 - a) without providing any derivative to the solver, and
 - b) by providing the derivatives and by estimating the state transition matrix with variational equations.
- 3) Considering the first guess solution in 1) and the procedure in Section 3.3, solve the problem with multiple shooting taking $N = 4$ and using the variational equation to compute the Jacobian of the nonlinear equality constraints.

(11 points)

1) The planar bicircular restricted four body dynamics model can be thought as a perturbation due to the Sun presence of the circular restricted three body problem considered in Exercise 1. In this case the normalized equations of motion can still be represented in a rotating frame aligned with the Earth-Moon direction as:

$$\begin{cases} \ddot{x} - 2\dot{y} = \frac{\partial U_4}{\partial x} \\ \ddot{y} + 2\dot{x} = \frac{\partial U_4}{\partial y} \end{cases} \quad \text{being the potential } U_4 = U_3 + \frac{m_s}{r_3(t)} - \frac{m_s}{\rho^2} (x \cos \omega_s t + y \sin \omega_s t) \quad (6)$$

Equation 6, already present in the paper but reported for clarity along other meaningful ones, shows that now the model is not autonomous anymore but explicitly depends on time because of the Sun presence.

The initial state $\mathbf{x}_0$, assumed always belonging to an Earth circular parking orbit, can be conveniently described in terms of two adimensional parameters only:

$$\mathbf{x}_0(\alpha, \beta) = \begin{pmatrix} x_0 \\ y_0 \\ \dot{x}_0 \\ \dot{y}_0 \end{pmatrix} = \begin{pmatrix} r_0 \cos \alpha - \mu \\ r_0 \sin \alpha \\ -(v_0 - r_0) \sin \alpha \\ (v_0 - r_0) \cos \alpha \end{pmatrix} \quad (7)$$

Furthermore the final transfer time is expressed as $t_f = t_i + \delta$. Using the values $\alpha = 1.5\pi$, $\beta = 1.41$, $t_i = 0$, $\delta = 7$ and the constants defined in the paper, the initial state coordinates become

$$\mathbf{x}_0 = (-0.01215, -0.01702, 10.72297, 0)^T$$

. Figure 5a represents the trajectory originating from the initial state in the rotating frame: since the constraints on the final position are not satisfied, it does not represent a solution of the transfer problem but can still be used as a first guess.

By applying at each time step the transformation reported in Equation 34 of the article we are referring to, the coordinates in the Earth-centered inertial frame can be obtained and are depicted in Figure 5b.

*F. Topputo, "On optimal two-impulse Earth-Moon transfers in a four-body model", *Celestial Mechanics and Dynamical Astronomy*, Vol. 117, pp. 279–313, 2013, DOI: 10.1007/s10569-013-9513-8.

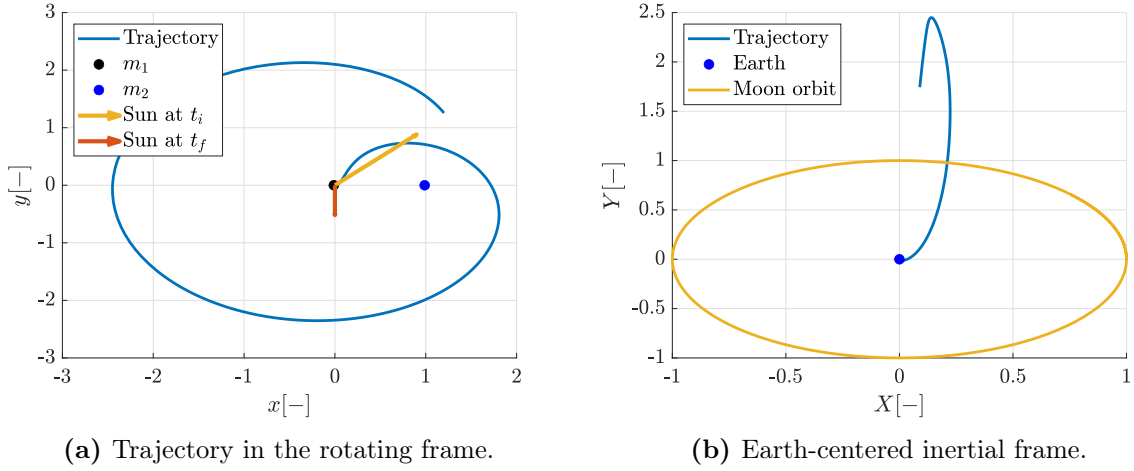


Figure 5: Trajectory resulting from propagation of the initial guess $\mathbf{x}_0$.

2) The optimization problem stated in Section 3.1 can be addressed in a first instance with a simple shooting method. Following this approach the NLP problem to be solved is stated as:

$$\min_{\mathbf{x}_i, t_i, t_f} J = \Delta v_i + \Delta v_f \quad \text{such that} \quad \begin{cases} \Psi_i(\mathbf{x}_i) = 0 \\ \Psi_f(\mathbf{x}_f) = 0 \\ t_i < t_f \end{cases} \quad (8)$$

being:

$$J = \Delta v_i + \Delta v_f = \|\mathbf{v}_1 - \mathbf{v}_E\| + \|\mathbf{v}_M - \mathbf{v}_f(\mathbf{x}_1)\|$$

The independent variables of Problem 8 are thus taken as the initial state $\mathbf{x}_i$, the initial time t_i , and the final time t_f . The state at final time is computed by means of the flow function $\mathbf{x}_f = \varphi(\mathbf{x}_i, t_i; t_f)$ and the nonlinear equality constraints Φ_i, Φ_f follow the definition given in the paper: they enforce a distance and a direction of the velocity vector at both Earth's and Moon's parking orbits.

The first option is to directly call MATLAB's solver `fmincon` to try and solve the problem without any additional information, however in this case we are able to provide the expression of the derivatives.

First, the State Transition Matrix is computed by integrating the variational equation:

$$\begin{cases} \dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}) \\ \dot{\Phi}(t_0, t) &= A\Phi(t_0, t) \\ \mathbf{x}(t_0) &= \mathbf{x}_0 \\ \Phi(t_0, t_0) &= I \end{cases} \quad \text{being} \quad A = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} 0 & I \\ W & 2\Omega \end{bmatrix} \quad (9)$$

where now all blocks are 2×2 square matrices and, referring to the matrix V defined in Equation 4:

$$\begin{aligned} W_{1,1} &= V_{1,1} - m_s/r_3^3 + 3m_s(x - \rho \cos \omega_s t)^2 / r_3^5 \\ W_{2,2} &= V_{2,2} - m_s/r_3^3 + 3m_s(y - \rho \sin \omega_s t)^2 / r_3^5 \\ W_{1,2} &= W_{2,1} = V_{1,2} + 3m_s(x - \rho \cos \omega_s t)(y - \rho \sin \omega_s t) / r_3^5 \end{aligned} \quad (10)$$

At this point the derivative of the cost function J with respect to the initial state $\mathbf{x}_1$ can be computed:

$$\frac{\partial J}{\partial \mathbf{x}_i} = \frac{\partial \Delta v_i}{\partial \mathbf{x}_i} + \frac{\partial \Delta v_f}{\partial \mathbf{x}_i} = \frac{\partial \Delta v_i}{\partial \mathbf{x}_i} + \frac{\partial \Delta v_f}{\partial \mathbf{x}_f} \cdot \frac{\partial \mathbf{x}_f}{\partial \mathbf{x}_i} = \frac{\partial \Delta v_i}{\partial \mathbf{x}_i} + \Phi^T \frac{\partial \Delta v_f}{\partial \mathbf{x}_f}$$

for which the derivatives $\frac{\partial \Delta v_i}{\partial \mathbf{x}_i}$ and $\frac{\partial \Delta v_f}{\partial \mathbf{x}_f}$ can be obtained analytically:

$$\frac{\partial \Delta v_i}{\partial \mathbf{x}_i} = \frac{1}{\sqrt{(\dot{x}_i - y_i)^2 + (\dot{y}_i + x_i + \mu)^2}} \begin{pmatrix} \dot{y}_i + x_i + \mu \\ -\dot{x}_i + y_i \\ \dot{x}_i - y_i \\ \dot{y}_i + x_i + \mu \end{pmatrix}$$

$$\frac{\partial \Delta v_f}{\partial \mathbf{x}_f} = \frac{1}{\sqrt{(\dot{x}_f - y_f)^2 + (\dot{y}_f + x_f + \mu - 1)^2}} \begin{pmatrix} \dot{y}_f + x_f + \mu - 1 \\ -\dot{x}_f + y_f \\ \dot{x}_f - y_f \\ \dot{y}_f + x_f + \mu - 1 \end{pmatrix}$$

While derivatives with respect to t_i and t_f are:

$$\frac{\partial J}{\partial t_i} = \frac{\partial \Delta v_i}{\partial t_i} + \frac{\partial \Delta v_f}{\partial t_i} = 0 + \frac{\partial \Delta v_f}{\partial \mathbf{x}_f} \cdot \frac{\partial \mathbf{x}_f}{\partial t_i} = -\mathbf{f}(\mathbf{x}_i, t_i)^T \Phi^T \frac{\partial \Delta v_f}{\partial \mathbf{x}_f}$$

$$\frac{\partial J}{\partial t_f} = \frac{\partial \Delta v_i}{\partial t_f} + \frac{\partial \Delta v_f}{\partial t_f} = 0 + \frac{\partial \Delta v_f}{\partial \mathbf{x}_f} \cdot \frac{\partial \mathbf{x}_f}{\partial t_f} = -\mathbf{f}(\varphi(\mathbf{x}_i, t_i; t_f), t_f)^T \frac{\partial \Delta v_f}{\partial \mathbf{x}_f}$$

So overall the quantities involved in the gradient of the performance index are known:

$$\nabla J = \begin{bmatrix} \frac{\partial J}{\partial \mathbf{x}_i} & \frac{\partial J}{\partial t_i} & \frac{\partial J}{\partial t_f} \end{bmatrix}$$

Lastly, the derivatives of the nonlinear equality constraints are computed as well. Concerning Ψ_i it results in a 2×6 vector that is derived analytically:

$$\begin{bmatrix} \frac{\partial \Psi_i}{\partial \mathbf{x}_i} & \frac{\partial \Psi_i}{\partial t_i} & \frac{\partial \Psi_i}{\partial t_f} \end{bmatrix} = \begin{bmatrix} 2(x_i + \mu) & 2y_i & 0 & 0 & 0 & 0 \\ \dot{x}_i & \dot{y}_i & x_i + \mu & y_i & 0 & 0 \end{bmatrix}$$

For what regards Ψ_f , it can be found that the gradient

$$\begin{bmatrix} \frac{\partial \Psi_f}{\partial \mathbf{r}_i} & \frac{\partial \Psi_f}{\partial \mathbf{v}_i} & \frac{\partial \Psi_f}{\partial t_i} & \frac{\partial \Psi_f}{\partial t_f} \end{bmatrix}$$

can be expressed explicitly as follows :

$$\begin{aligned} \frac{\partial \Psi_f}{\partial \mathbf{r}_i} &= \frac{\partial \Psi_f}{\partial \mathbf{r}_f} \frac{\partial \mathbf{r}_f}{\partial \mathbf{r}_i} = \frac{\partial \Psi_f}{\partial \mathbf{r}_f} \Phi_{rr} \\ \frac{\partial \Psi_f}{\partial \mathbf{v}_i} &= \frac{\partial \Psi_f}{\partial \mathbf{v}_f} \frac{\partial \mathbf{v}_f}{\partial \mathbf{v}_i} = \frac{\partial \Psi_f}{\partial \mathbf{v}_f} \Phi_{vv} \\ \frac{\partial \Psi_f}{\partial t_i} &= \frac{\partial \Psi_f}{\partial \mathbf{x}_f} \frac{\partial \mathbf{x}_f}{\partial t_i} = -\frac{\partial \Psi_f}{\partial \mathbf{x}_f} \Phi \mathbf{f}(\mathbf{x}_i, t_i) \\ \frac{\partial \Psi_f}{\partial t_f} &= \frac{\partial \Psi_f}{\partial \mathbf{x}_f} \frac{\partial \mathbf{x}_f}{\partial t_f} = \frac{\partial \Psi_f}{\partial \mathbf{x}_f} \mathbf{f}(\varphi(\mathbf{x}_i, t_i; t_f), t_f) \end{aligned}$$

with the known matrix:

$$\frac{\partial \Psi_f}{\partial \mathbf{x}_f} = \begin{bmatrix} \frac{\partial \Psi_f}{\partial \mathbf{r}_f} & \frac{\partial \Psi_f}{\partial \mathbf{v}_f} \end{bmatrix} = \begin{bmatrix} 2(x_f + \mu - 1) & 2y_f & 0 & 0 \\ \dot{x}_f & \dot{y}_f & x_f + \mu - 1 & y_f \end{bmatrix}$$

At this stage all the derivatives are available and the solver `fmincon` takes a great advantage if their expression is provided. By using the already defined values of $\mathbf{x}_0$, $t_i = 0$, and $t_f = \delta = 7$,

the problem has been solved twice: the first time by purely relying on the numerical capabilities of `fmincon`, the second one passing as input also the derivatives.

Despite the apparent similarity between the two solutions depicted in Figure 6, the outcomes have been extremely different. Having set the same tolerances for both cases, the first method did not really converge: it requires hundreds of iterations, thousands of function evaluations and consequently a noticeable amount of time. On the other hand, when the expression of the derivatives is available, the amount of effort required is significantly lower: it took only 23 iterations to find a suitable solution, which has been reported in Table 2.

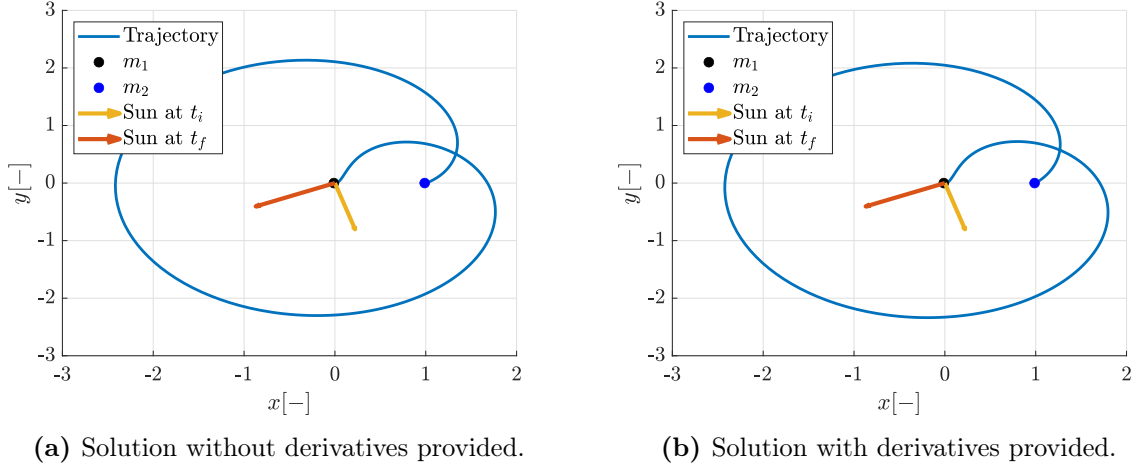


Figure 6: Comparison between the two solutions obtained for the simple shooting.

Table 2: Simple shooting solution.

J	x_i	y_i	$\dot{x}_i$	$\dot{y}_i$	t_i	t_f
4.215332	-0.0123659	-0.017024	10.721847	-0.135549	0.048151	7.845487

3) The multiple shooting method consists in dividing the trajectory into segments and modelling each node as a simple shooting. In this way the method’s robustness is increased, and it will suffer less from long times of flight, or from the presence of high nonlinearities. Taking N nodes and defining $\mathbf{x}_1, \mathbf{x}_N$ as the initial and final state respectively, with also $\mathbf{y} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N, t_i, t_f)$ as the vector of independent variables, the NLP problem statement becomes:

$$\min_{\mathbf{y}, t_i, t_f} J = \Delta v_1 + \Delta v_N \quad \text{such that} \quad \begin{cases} \mathbf{c}(\mathbf{y}) = 0 \\ \mathbf{g}(\mathbf{y}) < 0 \end{cases} \quad (11)$$

Problem 11 now has $4N + 2$ unknowns, $4N$ nonlinear equality constraints expressed by $\mathbf{c}(\mathbf{y}) = (\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_N, \Psi_1, \Psi_N)$, and $2N + 1$ nonlinear inequality constraints represented as $\mathbf{g}(\mathbf{y})$. The equations for these last two are defined in the reference paper.

The derivatives of the cost function and of the constraints can still be derived explicitly. Recalling that each state $\mathbf{x}_j$ is now independent, it is found that:

$$\nabla J = \begin{bmatrix} \frac{\partial J}{\partial \mathbf{x}_1} & 0 & \dots & 0 & \frac{\partial J}{\partial \mathbf{x}_N} & 0 & 0 \end{bmatrix}^T$$

$$\frac{\partial J}{\partial \mathbf{x}_1} = \frac{1}{\sqrt{(\dot{x}_1 - y_1)^2 + (\dot{y}_1 + x_1 + \mu)^2}} \begin{pmatrix} \dot{y}_1 + x_1 + \mu \\ y_1 - \dot{x}_1 \\ \dot{x}_1 - y_1 \\ \dot{y}_1 + x_1 + \mu \end{pmatrix}$$

$$\frac{\partial J}{\partial \mathbf{x}_N} = \frac{1}{\sqrt{(\dot{x}_N - y_N)^2 + (\dot{y}_N + x_N + \mu - 1)^2}} \begin{pmatrix} \dot{y}_N + x_N + \mu - 1 \\ y_N - \dot{x}_N \\ \dot{x}_N - y_N \\ \dot{y}_N + x_N + \mu - 1 \end{pmatrix}$$

Concerning the gradient of the equality constraints, which is a $4N \times 4N + 2$ matrix, it can be divided into four blocks, having different structures:

$$\frac{\partial \mathbf{c}}{\partial \mathbf{y}} = \begin{bmatrix} \frac{\partial \mathbf{z}}{\partial \mathbf{x}} & \frac{\partial \mathbf{z}}{\partial \mathbf{t}} \\ \frac{\partial \Psi}{\partial \mathbf{x}} & \frac{\partial \Psi}{\partial \mathbf{t}} \end{bmatrix} = \begin{bmatrix} M & P \\ Q & R \end{bmatrix} \quad (12)$$

M is given by the following $4(N-1) \times 4N$ square matrix:

$$\begin{bmatrix} \Phi(t_1; t_2) & -I & 0 & \dots & \dots & 0 \\ 0 & \Phi(t_2; t_3) & -I & 0 & \dots & 0 \\ \vdots & 0 & \ddots & \ddots & & 0 \\ 0 & \dots & \dots & 0 & \Phi(t_{N-1}; t_N) & -I \end{bmatrix} \quad \text{where } M_{j,k} = \frac{\partial \mathbf{z}_j}{\partial \mathbf{x}_k}$$

The contribution P has dimensions $4(N-1) \times 2$:

$$\begin{bmatrix} \frac{\partial \mathbf{z}_j}{\partial t_i} & \frac{\partial \mathbf{z}_j}{\partial t_f} \end{bmatrix} \quad \text{for } j = 1, \dots, N-1$$

$$\frac{\partial \mathbf{z}_j}{\partial t_i} = -\frac{N-j}{N-1} \Phi(t_j; t_{j+1}) \mathbf{f}(\mathbf{x}_j, t_j) + \mathbf{f}(\varphi(\mathbf{x}_j, t_j; t_{j+1}), t_{j+1}) \frac{N-j-1}{N-1}$$

$$\frac{\partial \mathbf{z}_j}{\partial t_f} = -\frac{j-1}{N-1} \Phi(t_j; t_{j+1}) \mathbf{f}(\mathbf{x}_j, t_j) + \mathbf{f}(\varphi(\mathbf{x}_j, t_j; t_{j+1}), t_{j+1}) \frac{j}{N-1}$$

Regarding the gradients of the constraints on initial and final states Ψ , the matrix R becomes a 4×2 null matrix $\mathbf{0}$, because neither Ψ_1, Ψ_N depends on t_i, t_f . The derivative with, respect to the states, however produces the $4 \times 4N$ matrix Q :

$$\frac{\partial \Psi}{\partial \mathbf{x}_j} = \begin{bmatrix} \frac{\partial \Psi_1}{\partial \mathbf{x}_k} \\ \frac{\partial \Psi_N}{\partial \mathbf{x}_k} \end{bmatrix} \quad \text{for } k = 1, \dots, N$$

being

$$\frac{\partial \Psi_1}{\partial \mathbf{x}_k} = \begin{bmatrix} 2(x_1 + \mu) & 2y_1 & 0 & 0 \\ \dot{x}_1 & \dot{y}_1 & x_1 + \mu & y_1 \end{bmatrix} \quad \text{if } k = 1 \quad \text{and} \quad \frac{\partial \Psi_1}{\partial \mathbf{x}_k} = 0 \text{ otherwise}$$

$$\frac{\partial \Psi_N}{\partial \mathbf{x}_k} = \begin{bmatrix} 2(x_N + \mu - 1) & 2y_N & 0 & 0 \\ \dot{x}_N & \dot{y}_N & x_N + \mu - 1 & y_N \end{bmatrix} \quad \text{if } k = N \quad \text{and} \quad \frac{\partial \Psi_N}{\partial \mathbf{x}_k} = 0 \text{ otherwise}$$

Now that ∇J and $\frac{\partial \mathbf{c}}{\partial \mathbf{y}}$ have been computed, they are provided to `fmincon` to solve the Problem 11 with $N = 4$. Figure 7a represents the trajectory of the initial guess given to the solver: it is computed by setting $\mathbf{x}_1 = \mathbf{x}_0$ and propagating it in time until $t_f^0 = t_i^0 + \delta = 0 + 7$. In Figure 7b instead there is the solution obtained from `fmincon` with the exact same tolerances set as in the simple shooting case.

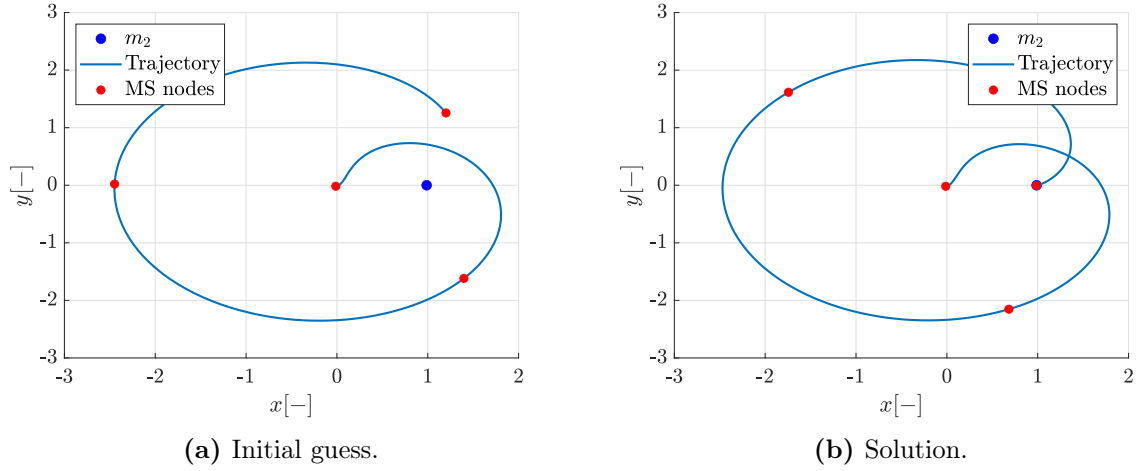


Figure 7: Multiple shooting results in the rotating frame.

Figure 8 and Table 3 compare different quantities characterizing the solutions obtained so far, when derivatives have been provided to the solver. It can be seen that the multiple shooting converges to a different state $\mathbf{y}_f$, characterized by a lower value of $J = \Delta v^{tot}$. This may be expected because, with respect to the single shooting, the method's robustness is improved, and it is an advantage when dealing with nonlinear dynamics, such as the one of the PBRFBP. At the same time the multiple shooting introduces more variables to deal with, therefore the computational effort required increases. On this subject, being able to provide to numerical solvers the expression of gradients, where possible, is of paramount importance; the amount of work to be done in advance is noticeable, but eventually it highly reduces computational times.

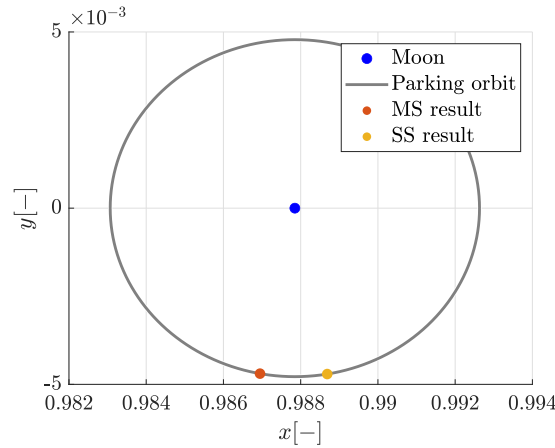


Figure 8: Simple shooting and multiple shooting arrival states.

Table 3: Comparison between simple and multiple shooting.

	J	t_i	t_f	iterations	CPU time [s]
SS	4.2153	0.0481	7.8454	23	3.0461
MS	4.0604	1.0033	9.2612	42	3.4799

3 Continuous guidance

Exercise 3

A low-thrust option is being considered for an Earth-Mars transfer[†]. Provide a *time-optimal* solution under the following assumptions: the spacecraft moves in the heliocentric two-body problem, Mars instantaneous acceleration is determined only by the Sun's gravitational attraction, the departure date is fixed, and the spacecraft initial and final states are coincident with those of the Earth and Mars, respectively.

- 1) Write down the spacecraft equations of motion, the costate dynamics, and the zero-finding problem for the unknowns $\{\lambda_0, t_f\}$ with the appropriate transversality condition.
- 2) Solve the problem considering the following data:
 - Launch date: 2022-08-03-12:45:20.000 UTC
 - Spacecraft mass: $m_0 = 1500$ kg
 - Electric properties: $T_{\max} = 150$ mN, $I_{sp} = 3000$ s
 - Number of thrusters: 4

Report the obtained solution in terms of $\{\lambda_0, t_f\}$ and the error with respect to the target. Validate your results exploiting the properties of time-optimal solutions.

- 3) Solve the problem for a degraded configuration with only 3 thrusters available, assuming that the failure occurs immediately after launch. Plot the thrust angles and compare them to the nominal case in 2).

(11 points)

1) The equations of motion modelling a two-body problem with a low-thrust action assumed, for which the spacecraft state is defined as $\mathbf{x} = (\mathbf{r}, \mathbf{v}, m)^T$, are:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, u, \hat{\alpha}) \text{ which explicitly is } \begin{cases} \dot{\mathbf{r}} = \mathbf{v} \\ \dot{\mathbf{v}} = -\frac{\mu}{r^3} \mathbf{r} + u \frac{T_{\max}}{m} \hat{\alpha} \\ \dot{m} = -u \frac{T_{\max}}{I_{sp} g_0} \end{cases} \quad (13)$$

where $u \in [0, 1]$ is the thrust throttle factor and $\hat{\alpha}$ is the unit vector representing the thrust pointing direction. Usually they are among the unknowns in trajectory optimization problems. Since they both have restrictions applied on them, the set of admissible control is defined:

$$U := \{(u, \boldsymbol{\alpha}) : 0 \leq u \leq 1; \|\boldsymbol{\alpha}\| = 1\}$$

The associated time-optimal problem, with conditions on the states given at both time extremes can be stated as:

$$\min_{U, t_f} J = \int_{t_i}^{t_f} l(\mathbf{x}, u, \hat{\alpha}, t) dt = \int_{t_i}^{t_f} 1 dt \text{ s.t. : } \begin{cases} \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, u, \hat{\alpha}) \\ \mathbf{x}_0(t_i) = \mathbf{x}_E \\ \mathbf{x}_f(t_f) = \mathbf{x}_M \end{cases} \quad (14)$$

The next step consists in introducing the vector of unknown functions $\boldsymbol{\lambda}$, called co-states, and defining the Hamiltonian function H that embeds both the functional and the constraints as:

[†]Read the necessary gravitational constants and planets positions from SPICE. Use the kernels provided on WeBeep for this assignment.

$$H(\mathbf{x}, \boldsymbol{\lambda}, u, \hat{\alpha}, t) = l(\mathbf{x}, u, \hat{\alpha}, t) + \boldsymbol{\lambda} \cdot \mathbf{f}(\mathbf{x}, u, \hat{\alpha}, t)$$

which, by recalling Equation 13 and 14, becomes:

$$H = l + \boldsymbol{\lambda}^T \mathbf{f} = 1 + \boldsymbol{\lambda}_r^T \dot{\mathbf{r}} + \boldsymbol{\lambda}_v^T \dot{\mathbf{v}} + \lambda_m \dot{m} = 1 + \boldsymbol{\lambda}_r^T \dot{\mathbf{r}} + -\frac{\mu}{r^3} \mathbf{r}^T \boldsymbol{\lambda}_v + \boldsymbol{\lambda}_v^T u \frac{T_{max}}{m} \hat{\alpha} + \lambda_m u \frac{T_{max}}{I_{sp} g_0} \quad (15)$$

Since the control actions $u, \hat{\alpha}$ are bounded, the solution will have to satisfy the necessary condition expressed by the Pontryagin Maximum Principle, which states:

$$(u, \hat{\alpha}) = \underset{(u, \hat{\alpha}) \in U}{\operatorname{argmin}} H(\mathbf{x}, \boldsymbol{\lambda}, u, \hat{\alpha}, t) \quad (16)$$

Therefore, the couple $(u, \hat{\alpha})$ that minimizes H at all times has to be found in order to fulfill Equation 16. To do so, focus is put on the last two terms of Equation 15. The common way to proceed is to define the Primer Vector:

$$\hat{\alpha} = -\frac{\boldsymbol{\lambda}_v}{\|\boldsymbol{\lambda}_v\|}$$

thus, the H function becomes:

$$H = 1 + \boldsymbol{\lambda}_r^T \dot{\mathbf{r}} + -\frac{\mu}{r^3} \mathbf{r}^T \boldsymbol{\lambda}_v + \frac{T_{max}}{I_{sp} g_0} u \left(-\lambda_v \frac{I_{sp} g_0}{m} - \lambda_m \right)$$

introducing the switching function St

$$St = -\lambda_v \frac{I_{sp} g_0}{m} - \lambda_m$$

the optimal control policy for the unknown u will then be:

$$u = \begin{cases} 1 & \text{if } St < 1 \\ 0 & \text{if } St > 1 \\ \text{in } [0, 1] & \text{if } St = 0 \end{cases}$$

Lastly, the equation ruling the co-states evolution is retrieved from the necessary condition

$$\dot{\boldsymbol{\lambda}} = -\frac{\partial H}{\partial \mathbf{x}}$$

that allows to write the system of differential equations associated to the time-optimal problem under study:

$$\begin{cases} \dot{\mathbf{r}} = \mathbf{v} \\ \dot{\mathbf{v}} = -\frac{\mu}{r^3} \mathbf{r} + u \frac{T_{max}}{m} \hat{\alpha} \\ \dot{m} = -u \frac{T_{max}}{I_{sp} g_0} \\ \dot{\lambda}_r = \frac{\mu}{r^3} \boldsymbol{\lambda}_v - 3 \frac{\mu}{r^5} (\boldsymbol{\lambda}_v^T \mathbf{r}) \mathbf{r} \\ \dot{\lambda}_v = -\dot{\lambda}_r \\ \dot{\lambda}_m = -\lambda_v u \frac{T_{max}}{m^2} \end{cases} \quad \text{with conditions} \quad \begin{cases} \mathbf{r}(t_0) = \mathbf{r}_E(t_0) \\ \mathbf{v}(t_0) = \mathbf{v}_E(t_0) \\ m(t_0) = m_0 \\ \mathbf{r}(t_f) = \mathbf{r}_M(t_f) \\ \mathbf{v}(t_f) = \mathbf{v}_M(t_f) \\ \lambda_m(t_f) = 0 \end{cases} \quad (17)$$

Equation 17 describes a TPBVP consisting of 14 ordinary differential equations and 14 boundary conditions, the last of which is introduced being the state m unknown at the final

time t_f . Being t_f a variable to be determined as well, the differential system has to be coupled with an additional condition of algebraic nature, called *transversality condition* that places a constraint on the final value of the Hamiltonian:

$$H(t_f) - \lambda_r^T \dot{\mathbf{x}}_M(t_f) - \lambda_v^T(t_f) \dot{\mathbf{v}}_M(t_f) = 0 \quad (18)$$

The solution to the time-optimal problem now consists in finding the 8 unknowns λ_0, t_f that satisfy both Equations 17 and 18. Once these values are obtained, the behaviour of $\mathbf{x}(t), \lambda(t)$ comes from numerical integration and the optimal control action $u, \hat{\alpha}$ are retrieved from the definition of the switching function and of the Primer Vector, respectively.

The TPBVP is usually solved by transforming it into a root-finding problem in the unknowns (λ_0, t_f) that states:

$$\text{Find } (\lambda_0, t_f) \text{ such that } \mathbf{F}(\lambda_0, t_f) = \begin{cases} \mathbf{r}(\mathbf{x}_0, \lambda_0, t_f) - \mathbf{r}_M(t_f) \\ \mathbf{v}(\mathbf{x}_0, \lambda_0, t_f) - \mathbf{v}_M(t_f) \\ \lambda_m(t_f) \\ H(t_f) - \lambda_r^T \dot{\mathbf{x}}_M(t_f) - \lambda_v^T(t_f) \dot{\mathbf{v}}_M(t_f) \end{cases} = \mathbf{0} \quad (19)$$

2) Problem 19 is solved in the nominal condition scenario by means of the MATLAB solver `fsolve`. In order to get a more stable numerical behaviour the units have been switched from the triplet [km,s,kg] to [AU,day,kg]. The heliocentric gravitational parameter μ and the two planets' states have been retrieved with proper SPICE routines.

The first guess for t_f has been obtained by considering that a low-thrust propelled interplanetary transfer will likely take a good amount of time: as a consequence, the initial value for the transfer duration has been set to approximately 6 months.

Concerning the guess of λ_0 , its value has been selected after several tries in which the orders of magnitude of the λ_r, λ_v were changed one at a time.

A working initial guess has thus been found to be:

$$\begin{aligned} \lambda_{r_0}^{guess} &= (1, 1, 1)^T \cdot 10^2 \\ \lambda_{v_0}^{guess} &= -(1, 1, 1)^T \cdot 10^5 \\ \lambda_{m_0}^{guess} &= 50 \end{aligned}$$

Table 4 lists the initial values of $\lambda_r, \lambda_v, t_f$ associated to the solution, while the errors in the final state are reported in Table 5. Furthermore, a visual representation of the transfer trajectories is reported in Figure 9.

$\lambda_{0,r}$ [day/AU]	-314.4792	226.5844	15.6092
$\lambda_{0,v}$ [day ² /AU]	-23497.7244	-10947.6524	-303.7489
$\lambda_{0,m}$ [day/kg]	0.0371		
t_f	2023-05-12-08:57:55.880 UTC		
TOF [days]	281.8420		

Table 4: Time-optimal Earth-Mars transfer solution in nominal condition.

$\ \mathbf{r}_f(t_f) - \mathbf{r}_M(t_f)\ $	[km]	0.004
$\ \mathbf{v}_f(t_f) - \mathbf{v}_M(t_f)\ $	[km/s]	$6.085 \cdot 10^{-6}$

Table 5: Final state error with respect to Mars' center in nominal conditions.

Figure 10 shows the functions $St(t), u(t), m(t)$ associated to the obtained transfer trajectory. It can be observed that, being the switching function always negative during the flight,

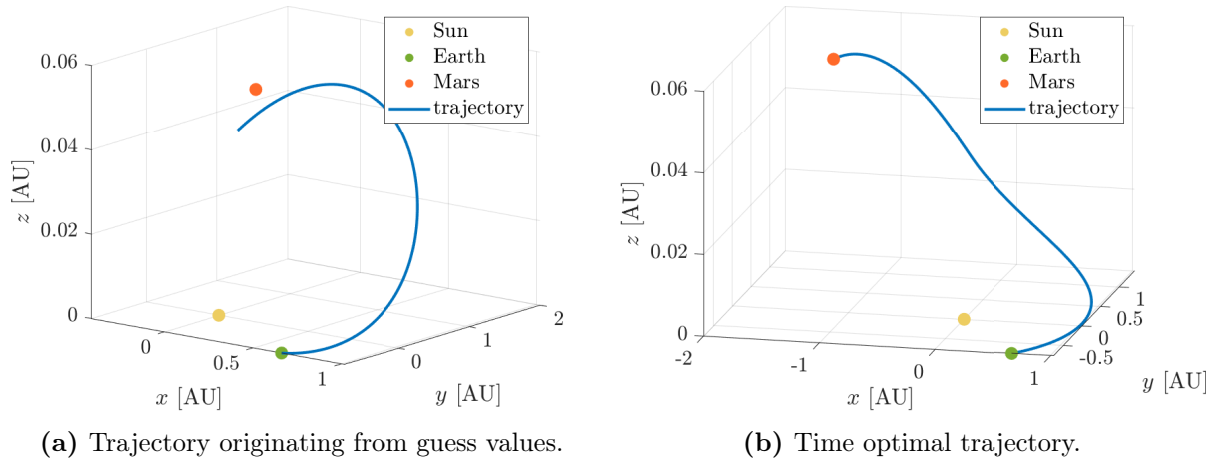


Figure 9: Trajectories in Sun centered inertial frame.

then the thrust factor is always equal to 1. This was expected, being as a general rule the thrust always at the maximum possible levels in time-optimal transfers.

Lastly, in Figure 11 the evolution of the thrust direction vector $\hat{\alpha}$ can be observed. Initially, it acts outwards in order to enlarge the position magnitude, whereas at a specific point there is an inversion: thrust is used to align the transfer trajectory towards Mars' orbit.

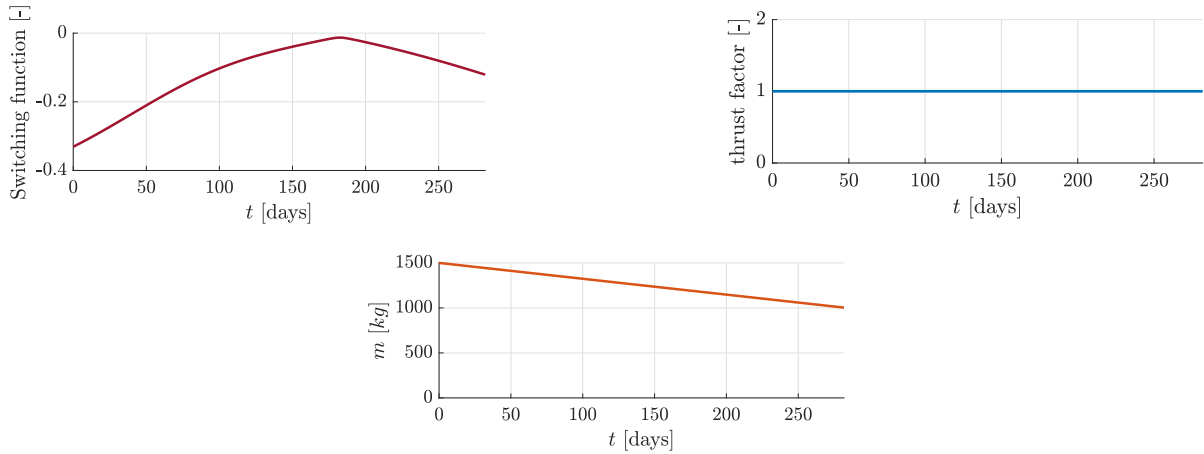


Figure 10: Time evolution of thrust related quantities in the time-optimal solution.

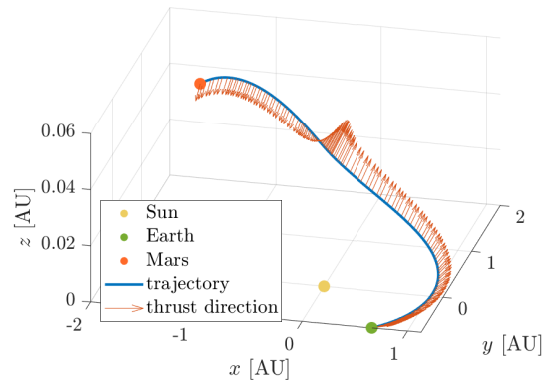


Figure 11: Thrust direction during transfer in nominal conditions.

3) In order to solve the time-optimal problem for the degraded configuration, at first the idea was to use the solutions obtained above as first guess candidates. However, a satisfactory convergence of the numerical method could not be achieved. To overcome the issue, a new triplet of units has been introduced: [AU, year, –]. Also in this case a global minimum could not be located, being the solver almost stuck in an almost flat region. These experiences lead to the thought that a reduction in the maximum available thrust, even if not dramatic, may completely change the shape of the solution: as a matter of fact, the working initial guess has been found by setting a transfer time of 2 years. The associated vectors that make **fsolve** converge are:

$$\begin{aligned}\lambda_{r_0}^{guess} &= (1, 1, 10)^T \\ \lambda_{v_0}^{guess} &= -(1, 1, 10)^T \\ \lambda_{m_0}^{guess} &= 1\end{aligned}$$

Figure 12 proofs that solution of the transfer largely differs from the one obtained in the previous scenario. In Table 6 and 7 it can be observed how the time required to reach Mars has more than doubled.

Once again, as shown in Figure 13, the spacecraft mass decreases continuously, being the thrusters always active. Lastly, Figure 14 describes how the thrust direction vector changes along the trajectory: its orientation keeps changing, confirming how the solution has grown in complexity.

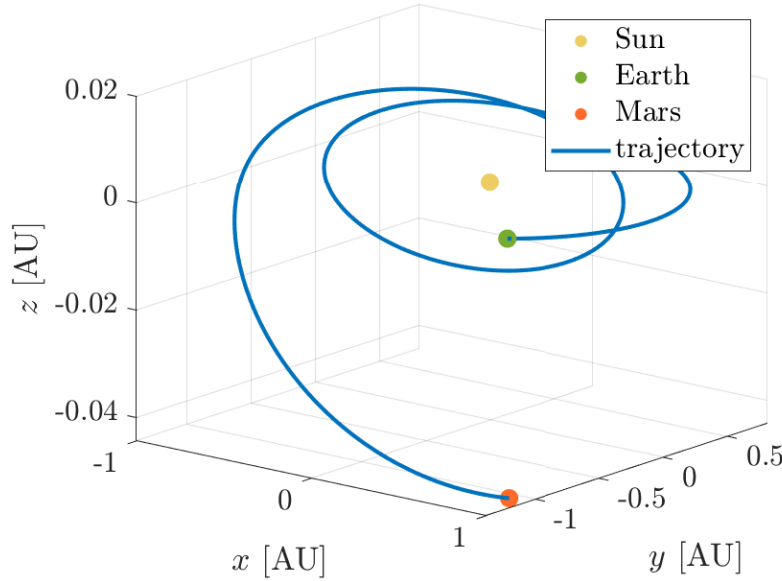


Figure 12: Time-optimal transfer in degraded conditions.

$\lambda_{0,r}$ [s/km]	0.3288	-0.1894	0.0059
$\lambda_{0,v}$ [year ² /AU]	0.0622	0.0243	-0.00029
$\lambda_{0,m}$ [year/-]	0.2045		
t_f	2024-03-27-22:35:48.521 UTC		
TOF [days]	602.4100		

Table 6: Time-optimal Earth-Mars transfer solution in degraded condition.

$\ \mathbf{r}_f(t_f) - \mathbf{r}_M(t_f)\ $	[km]	0.001
$\ \mathbf{v}_f(t_f) - \mathbf{v}_M(t_f)\ $	[km/s]	$4.888 \cdot 10^{-8}$

Table 7: Final state error with respect to Mars' center in degraded conditions.

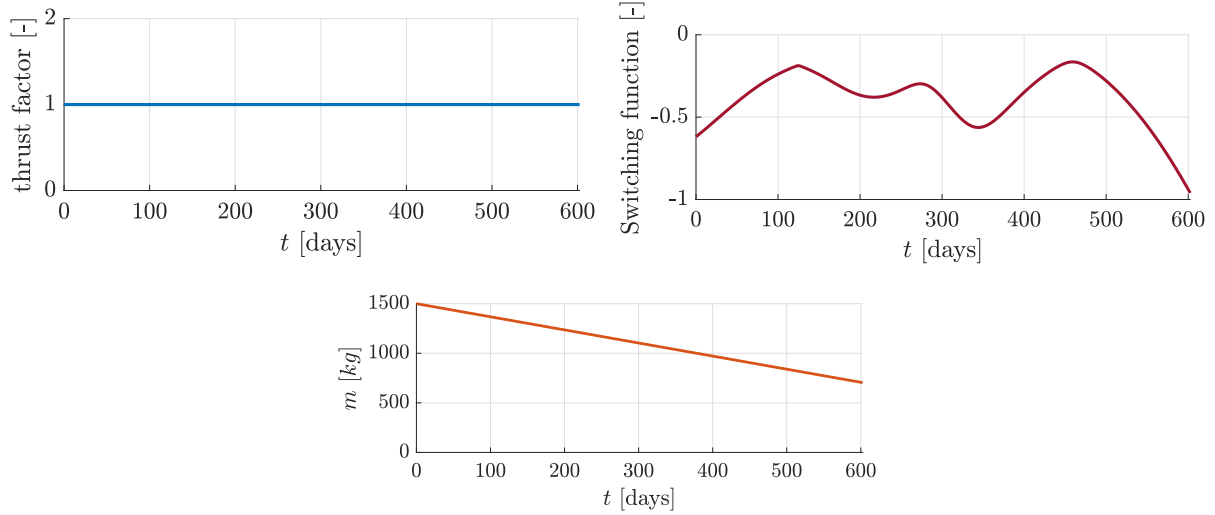


Figure 13: Time evolution of thrust related quantities in the degraded time-optimal solution.

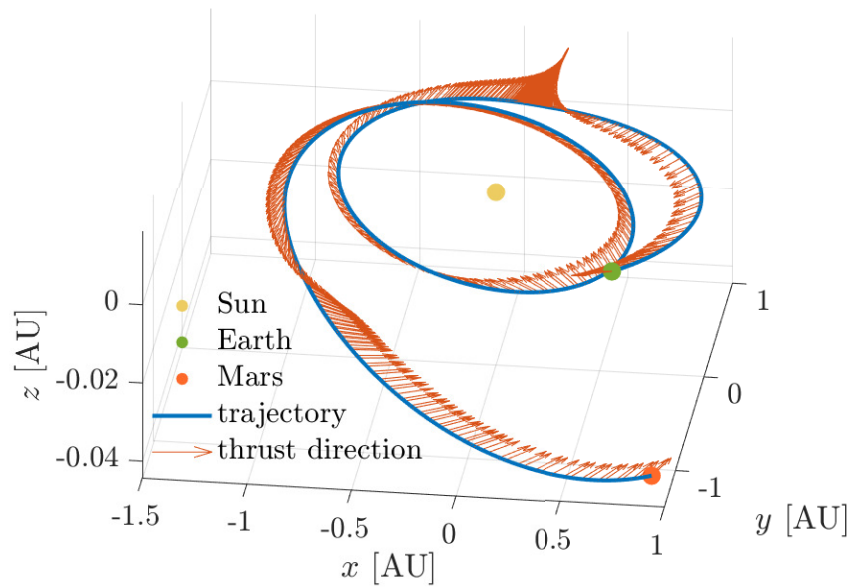


Figure 14: Thrust direction during transfer in degraded conditions.